

Comprehensive Mathematical Formula Report: JPMLCOE Project

February 25, 2026

Contents

1	General State-Space Model Framework	2
1.1	Basic State-Space Model Definition	2
1.2	Bayesian Filtering Equations	2
1.3	Log-Likelihood for Parameter Inference	2
2	State-Space Models	2
2.1	Linear-Gaussian Model	2
2.2	Cubic Sensor Model	3
2.3	Stochastic Volatility Model	3
2.4	Kitagawa Model	4
2.5	Range-Bearing Model	4
2.6	Acoustic Tracking Model	4
2.7	Lorenz 96 Model	5
2.8	Two-Sensor Bearing Model	5
3	Kalman Filter Family	5
3.1	Standard Kalman Filter (KF)	5
3.2	Extended Kalman Filter (EKF)	6
3.3	Unscented Kalman Filter (UKF)	6
4	Particle Filters	7
4.1	Bootstrap Particle Filter	7
4.2	Conditional SMC	7
5	Deterministic Particle Flow Filters	8
5.1	Exact Daum-Huang (EDH) Flow	8
5.2	Local EDH (LEDH) Flow	8
5.3	LEDH Invertible Particle Flow Filter (Algorithm 1)	9
6	Importance Weights for Invertible Flows	9
6.1	Full Importance Weight Formula	9

7	Stochastic Particle Flow Filters	9
7.1	Stochastic EDH Flow	9
7.2	SDE with Local Correction	10
8	Stiffness Mitigation via Optimal Schedule	10
8.1	BVP for Optimal Homotopy Schedule	10
8.2	Jacobian Stiffness Ratio Schedule	10
9	Kernel Mapping Particle Flow Filter	11
9.1	Matrix-Valued Kernel	11
9.2	Scalar Kernel	11
9.3	Kernel Divergence	11
9.4	Flow Update (SVGD-like)	11
9.5	Localization Matrix (Cyclic Domain)	11
10	Resampling Methods	12
10.1	Systematic Resampling	12
10.2	Soft Resampling	12
10.3	Entropy-Regularized Optimal Transport	12
11	Parameter Inference	12
11.1	HMC for Differentiable Filters	12
11.2	Parameter Bijectors	13
11.3	PMMH (Particle Marginal Metropolis-Hastings)	13
11.4	Particle Gibbs + HMC	14
12	Numerical Linear Algebra	14
12.1	Safe Cholesky	14
12.2	Safe Log-Absolute Determinant	14
12.3	Matrix Square Root	14
12.4	Symmetrize	14
13	ODE/SDE Solvers	15
13.1	Euler Method	15
13.2	Runge-Kutta 4	15
13.3	Euler-Maruyama (SDE)	15
14	LaTeX Report – Additional Derivations	15
14.1	Importance Sampling Discretization	15
14.2	KL Divergence for Kernel Flow	15
14.3	Entropy-Regularized Wasserstein Distance	16
14.4	Joseph Form Limiting Behavior	16
14.5	Fokker-Planck Equation	16
15	Summary of Formulas Requiring Hand Derivation	16

1 General State-Space Model Framework

1.1 Basic State-Space Model Definition

Files: code/src/core/model_base.py, report/main.tex

State transition (discrete-time):

$$x_k = f(x_{k-1}, w_k), \quad w_k \sim \mathcal{N}(0, Q) \quad (1)$$

Observation model:

$$z_k = h(x_k, u_k), \quad u_k \sim \mathcal{N}(0, R) \quad (2)$$

1.2 Bayesian Filtering Equations

File: report/main.tex

Prediction step:

$$p(x_k | z_{1:k-1}) = \int p(x_k | x_{k-1}) p(x_{k-1} | z_{1:k-1}) dx_{k-1} \quad (3)$$

Update step (Bayes rule):

$$p(x_k | z_{1:k}) = \frac{p(z_k | x_k) p(x_k | z_{1:k-1})}{p(z_k | z_{1:k-1})} \quad (4)$$

Marginal likelihood:

$$p(z_k | z_{1:k-1}) = \int p(z_k | x_k) p(x_k | z_{1:k-1}) dx_k \quad (5)$$

1.3 Log-Likelihood for Parameter Inference

Negative log-likelihood (for MLE):

$$L(\theta) = - \sum_{k=1}^T \log p_{\theta}(z_k | z_{1:k-1}) \quad (6)$$

MAP objective (with prior):

$$L_{\text{MAP}}(\theta) = L(\theta) - \log p(\theta) \quad (7)$$

2 State-Space Models

2.1 Linear-Gaussian Model

File: code/src/models/linear_gaussian.py

State transition:

$$X_n = F X_{n-1} + B V_n, \quad V_n \sim \mathcal{N}(0, I) \quad (8)$$

Observation:

$$Y_n = HX_n + DW_n, \quad W_n \sim \mathcal{N}(0, I) \quad (9)$$

Noise covariances:

$$Q = BB^\top \quad (10)$$

$$R = DD^\top \quad (11)$$

Log observation probability:

$$\log p(y|x) = -\frac{1}{2} (d \log(2\pi) + \log |R| + (y - Hx)^\top R^{-1} (y - Hx)) \quad (12)$$

2.2 Cubic Sensor Model

File: code/src/models/cubic_sensor.py

State transition:

$$x_t = ax_{t-1} + v_t, \quad v_t \sim \mathcal{N}(0, \sigma_v^2) \quad (13)$$

Observation:

$$y_t = x_t + cx_t^3 + w_t, \quad w_t \sim \mathcal{N}(0, \sigma_w^2) \quad (14)$$

Observation Jacobian:

$$\frac{dh}{dx} = 1 + 3cx^2 \quad (\text{always positive}) \quad (15)$$

2.3 Stochastic Volatility Model

File: code/src/models/stochastic_volatility.py

State transition:

$$x_t = \alpha x_{t-1} + \sigma w_t, \quad w_t \sim \mathcal{N}(0, 1) \quad (16)$$

Observation:

$$y_t = \beta e^{x_t/2} v_t, \quad v_t \sim \mathcal{N}(0, 1) \quad (17)$$

Stationary variance:

$$\text{Var}(x_\infty) = \frac{\sigma^2}{1 - \alpha^2} \quad (18)$$

Observation variance:

$$\text{Var}(y_t|x_t) = \beta^2 e^{x_t} \quad (19)$$

Observation Jacobian:

$$\frac{dh}{dx} = 0 \quad (\text{causes EKF/UKF failure}) \quad (20)$$

2.4 Kitagawa Model

File: `code/src/models/kitagawa.py`

State transition:

$$x_n = \frac{1}{2}x_{n-1} + \frac{25x_{n-1}}{1 + x_{n-1}^2} + 8 \cos(1.2n) + v_n \quad (21)$$

Observation:

$$y_n = \frac{x_n^2}{20} + w_n \quad (22)$$

State Jacobian:

$$\frac{df}{dx} = \frac{1}{2} + \frac{25(1 - x^2)}{(1 + x^2)^2} \quad (23)$$

Observation Jacobian:

$$\frac{dh}{dx} = \frac{x}{10} \quad (24)$$

2.5 Range-Bearing Model

File: `code/src/models/range_bearing.py`

Observation functions:

$$\text{range} = \sqrt{(x - x_s)^2 + (y - y_s)^2} \quad (25)$$

$$\text{bearing} = \text{atan2}(y - y_s, x - x_s) \quad (26)$$

Observation Jacobian:

$$H = \begin{bmatrix} \frac{dx}{r} & \frac{dy}{r} & 0 & 0 \\ -\frac{dy}{r^2} & \frac{dx}{r^2} & 0 & 0 \end{bmatrix} \quad (27)$$

where $dx = x - x_s$, $dy = y - y_s$, $r = \text{range}$.

2.6 Acoustic Tracking Model

File: `code/src/models/acoustic_tracking.py`

State (4D constant velocity):

$$F = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (28)$$

Observation (amplitude decay):

$$z_j = \frac{\Psi}{r_j^2 + d_0} + \text{noise} \quad (29)$$

Observation Jacobian:

$$\frac{\partial h_j}{\partial x} = -\frac{2\Psi(x - s_{x,j})}{(r_j^2 + d_0)^2} \quad (30)$$

2.7 Lorenz 96 Model

File: `code/src/models/lorenz96.py`

Dynamics (cyclic boundary):

$$\frac{dx_i}{dt} = (x_{i+1} - x_{i-2})x_{i-1} - x_i + F \quad (31)$$

Numerical Jacobian (centered differences):

$$\left(\frac{df}{dx}\right)_{ij} \approx \frac{f_i(x + \delta e_j) - f_i(x - \delta e_j)}{2\delta} \quad (32)$$

2.8 Two-Sensor Bearing Model

File: `code/src/models/two_sensor_bearing.py`

Observation:

$$\text{bearing}_i = \text{atan2}(y - y_{s,i}, x - x_{s,i}) + v_i \quad (33)$$

Observation Jacobian:

$$\frac{\partial(\text{bearing}_i)}{\partial x} = -\frac{y - y_{s,i}}{r_i^2} \quad (34)$$

$$\frac{\partial(\text{bearing}_i)}{\partial y} = \frac{x - x_{s,i}}{r_i^2} \quad (35)$$

3 Kalman Filter Family

3.1 Standard Kalman Filter (KF)

File: `code/src/filters/kalman/kalman.py`

Prediction:

$$\mathbf{m}_{\text{pred}} = F\mathbf{m} \quad (36)$$

$$P_{\text{pred}} = FPF^\top + Q \quad (37)$$

Innovation covariance:

$$S = HP_{\text{pred}}H^\top + R \quad (38)$$

Kalman gain:

$$K = P_{\text{pred}}H^\top S^{-1} \quad (39)$$

Joseph form covariance update (numerically stable):

$$P_{\text{updated}} = (I - KH)P_{\text{pred}}(I - KH)^\top + KRK^\top \quad (40)$$

Log-likelihood per step:

$$\log p(z_k | z_{1:k-1}) = -\frac{1}{2} (\log |2\pi S| + \nu^\top S^{-1} \nu) \quad (41)$$

where $\nu = z_k - H\mathbf{m}_{\text{pred}}$ is the innovation.

3.2 Extended Kalman Filter (EKF)

File: `code/src/filters/kalman/extended_kalman.py`

Linearization:

$$f(x) \approx f(\hat{x}) + F(x - \hat{x}), \quad F = \left. \frac{\partial f}{\partial x} \right|_{\hat{x}} \quad (42)$$

$$h(x) \approx h(\hat{x}) + H(x - \hat{x}), \quad H = \left. \frac{\partial h}{\partial x} \right|_{\hat{x}} \quad (43)$$

Prediction:

$$\mathbf{m}_{\text{pred}} = f(\mathbf{m}) \quad (44)$$

$$P_{\text{pred}} = F P F^\top + Q \quad (45)$$

Update (via Cholesky):

$$L_S K^\top = H P_{\text{pred}} \implies K = (L_S^{-\top} (H P_{\text{pred}})^\top)^\top \quad (46)$$

Mean and covariance update:

$$\mathbf{m}_{\text{updated}} = \mathbf{m}_{\text{pred}} + K(\mathbf{z} - h(\mathbf{m}_{\text{pred}})) \quad (47)$$

$$P_{\text{updated}} = (I - KH)P_{\text{pred}}(I - KH)^\top + KRK^\top \quad (48)$$

3.3 Unscented Kalman Filter (UKF)

File: `code/src/filters/kalman/unscented_kalman.py`

Sigma point parameters:

$$\lambda = \alpha^2(n + \kappa) - n \quad (49)$$

Weights for mean:

$$W_m^{(0)} = \frac{\lambda}{n + \lambda} \quad (50)$$

$$W_m^{(i)} = \frac{1}{2(n + \lambda)}, \quad i = 1, \dots, 2n \quad (51)$$

Weights for covariance:

$$W_c^{(0)} = W_m^{(0)} + (1 - \alpha^2 + \beta) \quad (52)$$

$$W_c^{(i)} = \frac{1}{2(n + \lambda)}, \quad i = 1, \dots, 2n \quad (53)$$

Sigma point generation:

$$\mathbf{x}^{(0)} = \boldsymbol{\mu} \quad (54)$$

$$\mathbf{x}^{(i)} = \boldsymbol{\mu} + \sqrt{(n + \lambda)P_i}, \quad i = 1, \dots, n \quad (55)$$

$$\mathbf{x}^{(n+i)} = \boldsymbol{\mu} - \sqrt{(n + \lambda)P_i}, \quad i = 1, \dots, n \quad (56)$$

Prediction:

$$\boldsymbol{\chi}_{\text{pred}}^{(i)} = f(\boldsymbol{\chi}^{(i)}) \quad (57)$$

$$\boldsymbol{\mu}_{\text{pred}} = \sum_i W_m^{(i)} \boldsymbol{\chi}_{\text{pred}}^{(i)} \quad (58)$$

$$\Sigma_{\text{pred}} = \sum_i W_c^{(i)} (\boldsymbol{\chi}_{\text{pred}}^{(i)} - \boldsymbol{\mu}_{\text{pred}})(\boldsymbol{\chi}_{\text{pred}}^{(i)} - \boldsymbol{\mu}_{\text{pred}})^\top + Q \quad (59)$$

4 Particle Filters

4.1 Bootstrap Particle Filter

File: code/src/filters/particle/bootstrap_pf_tf.py

Weight update:

$$\log w_t^{(i)} = \log w_{t-1}^{(i)} + \log p(z_t | x_t^{(i)}) \quad (60)$$

Normalized weights (log-sum-exp):

$$w^{(i)} = \frac{\exp(\log w^{(i)} - \max_j \log w^{(j)})}{\sum_k \exp(\log w^{(k)} - \max_j \log w^{(j)})} \quad (61)$$

Effective Sample Size:

$$\text{ESS} = \frac{1}{\sum_i (w_i)^2} \quad (62)$$

Weighted mean and covariance:

$$\mathbf{m}_t = \sum_i w_t^{(i)} x_t^{(i)} \quad (63)$$

$$P_t = \sum_i w_t^{(i)} (x_t^{(i)} - \mathbf{m}_t)(x_t^{(i)} - \mathbf{m}_t)^\top \quad (64)$$

4.2 Conditional SMC

File: code/src/filters/particle/conditional_smc.py

Ancestor sampling (optional):

$$\log w_{t|T}^{(j)} = \log w_{t-1}^{(j)} + \log \mathcal{N}(x_t^*, f(x_{t-1}^{(j)}), Q) \quad (65)$$

Trajectory sampling:

$$k_T \sim \text{Categorical}(w_T) \quad (66)$$

$$k_t = \text{ancestors}_{t+1}[k_{t+1}], \quad t = T-1, \dots, 0 \quad (67)$$

5 Deterministic Particle Flow Filters

5.1 Exact Daum-Huang (EDH) Flow

Files: code/src/filters/particle/edh_flow.py, code/src/utils/flow_params.py

Core flow ODE:

$$\frac{d\eta}{d\lambda} = A(\lambda)\eta + b(\lambda), \quad \lambda \in [0, 1] \quad (68)$$

Flow parameters (Eq. 10-11 from Li & Coates 2017):

$$A(\lambda) = -\frac{1}{2}PH^\top(\lambda HPH^\top + R)^{-1}H \quad (69)$$

Nonlinear correction:

$$e = h(x) - Hx \quad (70)$$

Drift vector:

$$b(\lambda) = (I + 2\lambda A) [(I + \lambda A)PH^\top R^{-1}(z - e) + A\bar{\eta}_0] \quad (71)$$

Exponential lambda schedule (Li & Coates 2017):

$$q = 1.2 \quad (72)$$

$$\varepsilon_1 = \frac{1 - q}{1 - q^n} \quad (73)$$

$$d\lambda_k = \varepsilon_1 q^k, \quad k = 0, 1, \dots, n - 1 \quad (74)$$

Euler integration:

$$\eta(\lambda + d\lambda) = \eta(\lambda) + d\lambda(A(\lambda)\eta(\lambda) + b(\lambda)) \quad (75)$$

5.2 Local EDH (LEDH) Flow

File: code/src/filters/particle/ledh_flow.py

Per-particle local linearization:

$$H_i = \left. \frac{\partial h}{\partial x} \right|_{x_i} \quad (76)$$

Local flow parameters:

$$A_i(\lambda) = -\frac{1}{2}PH_i^\top(\lambda H_iPH_i^\top + R)^{-1}H_i \quad (77)$$

With optional regularization:

$$P_{\text{reg}} = P + \text{reg} \cdot \frac{\text{trace}(P)}{n} I \quad (78)$$

5.3 LEDH Invertible Particle Flow Filter (Algorithm 1)

File: `code/src/filters/particle/ledh_invertible.py`

Jacobian tracking:

$$M_i = I + d\lambda \cdot A_i \quad (79)$$

Log-determinant accumulation:

$$\log \theta_i \leftarrow \log \theta_i + \log |\det(M_i)| \quad (80)$$

6 Importance Weights for Invertible Flows

6.1 Full Importance Weight Formula

File: `code/src/utis/distributions.py`

$$\log w_i = \log p(z|\eta_1^i) + \log p(\eta_1^i|x_{k-1}^i) + \log \theta_i - \log p(\eta_0^i|x_{k-1}^i) + \log w_{k-1}^i \quad (81)$$

Observation likelihood:

$$\log p(z|\eta) = -\frac{1}{2}(n \log(2\pi) + \log |R| + (z - h(\eta))^\top R^{-1}(z - h(\eta))) \quad (82)$$

Transition likelihood (via Cholesky):

$$L_Q = \text{cholesky}(Q) \quad (83)$$

$$\log p(\eta|x_{k-1}) = -\frac{1}{2}(d \log(2\pi) + 2 \sum_i \log(L_{i,i}) + \|L_Q^{-1}(\eta - f(x_{k-1}))\|^2) \quad (84)$$

7 Stochastic Particle Flow Filters

7.1 Stochastic EDH Flow

File: `code/src/filters/particle/stochastic_edh.py`

SDE extension (Dai & Daum 2021, arXiv:2103.09676):

$$dx = [A(\lambda)x + b(\lambda)]d\lambda + \sqrt{Q} dW_\lambda \quad (85)$$

where $Q = \text{diag}(q_1, \dots, q_d)$ is diagonal diffusion.

Score-corrected drift:

$$f_{\text{stoch}}(x, \lambda) = f_{\text{det}}(x, \lambda) + \frac{1}{2}Q \nabla_x \log p(x, \lambda) \quad (86)$$

Modified flow matrices:

$$A = A_{\text{det}} - \frac{Q}{2} \Sigma(\lambda)^{-1} \quad (87)$$

$$b = b_{\text{det}} + \frac{Q}{2} \Sigma(\lambda)^{-1} m(\lambda) \quad (88)$$

Precision matrix and information vector:

$$\Sigma(\lambda)^{-1} = P_0^{-1} + \lambda H^\top R^{-1} H \quad (89)$$

$$\Sigma(\lambda)^{-1} m(\lambda) = P_0^{-1} \bar{\eta}_0 + \lambda H^\top R^{-1} y \quad (90)$$

Euler-Maruyama:

$$x \leftarrow x + [Ax + b]d\lambda + \sqrt{Q_{\text{diag}} \cdot d\lambda} dW \quad (91)$$

Fokker-Planck equation:

$$\frac{\partial p}{\partial \lambda} = -\nabla \cdot (fp) + \frac{1}{2} \nabla \cdot (QQ^\top \nabla p) \quad (92)$$

7.2 SDE with Local Correction

File: code/src/filters/particle/sde-local-correction.py

Linearization error:

$$e_0 = h(\bar{\eta}_0) - H\bar{\eta}_0 \quad (93)$$

Corrected observation:

$$z_{\text{corrected}} = z - e_0 \quad (94)$$

8 Stiffness Mitigation via Optimal Schedule

8.1 BVP for Optimal Homotopy Schedule

File: code/src/filters/particle/stochastic-edh.py

Boundary Value Problem:

$$\beta'' = \mu \frac{\partial \kappa}{\partial \beta} \quad (95)$$

$$\beta(0) = 0, \quad \beta(1) = 1 \quad (96)$$

Nuclear-norm condition number:

$$\kappa_\nu(M) = \text{tr}(M) \cdot \text{tr}(M^{-1}) \quad (97)$$

where $M(\beta) = P_0^{-1} + \beta H^\top R^{-1} H$.

Derivative:

$$\frac{\partial \kappa_\nu}{\partial \beta} = \text{tr}(J_{\text{meas}}) \text{tr}(M^{-1}) - \text{tr}(M) \text{tr}(M^{-1} J_{\text{meas}} M^{-1}) \quad (98)$$

8.2 Jacobian Stiffness Ratio Schedule

File: code/src/filters/particle/stochastic-edh.py

Flow Jacobian:

$$F(\beta) = -\frac{1}{2} M(\beta)^{-1} J_{\text{meas}} - \text{diag}(Q/2) M(\beta) \quad (99)$$

Stiffness ratio:

$$R_{\text{stiff}} = \frac{\max_i |\Re(\lambda_i(F))|}{\min_i |\Re(\lambda_i(F))|} \quad (100)$$

9 Kernel Mapping Particle Flow Filter

9.1 Matrix-Valued Kernel

File: code/src/filters/particle/kernel_flow.py

Per-dimension Gaussian kernel:

$$K^{(d)}(x_i, x_j) = \exp \left(-\frac{1}{2} \frac{(x_i^{(d)} - x_j^{(d)})^2}{\alpha \sigma_d^2} \right) \quad (101)$$

9.2 Scalar Kernel

File: code/src/filters/particle/kernel_flow.py

Mahalanobis distance variant:

$$A = \frac{(\alpha B)^{-1}}{d} \quad (102)$$

$$K(x_i, x_j) = \exp \left(-\frac{1}{2} (x_i - x_j)^\top A (x_i - x_j) \right) \quad (103)$$

9.3 Kernel Divergence

File: code/src/filters/particle/kernel_flow.py

$$\nabla_{x_i} K = \frac{\text{diff}}{\text{scale}} \cdot K \quad (104)$$

9.4 Flow Update (SVGD-like)

File: code/src/filters/particle/kernel_flow.py

$$I_{f,i} = \frac{1}{N} \left[\sum_j K(x_i, x_j) \nabla \log p_{\text{post}}(x_j) + \sum_j \nabla_j K(x_i, x_j) \right] \quad (105)$$

Posterior gradient:

$$\nabla \log p_{\text{post}} = H^\top R^{-1} (y - h(x)) - B^{-1} (x - x_0) \quad (106)$$

9.5 Localization Matrix (Cyclic Domain)

File: code/src/filters/particle/kernel_flow.py

$$C_{ij} = \exp \left(- \left(\frac{d(i, j)}{r_{\text{in}}} \right)^2 \right) \quad (107)$$

Schur product localization:

$$B_{\text{localized}} = B_{\text{sample}} \odot C \quad (108)$$

10 Resampling Methods

10.1 Systematic Resampling

File: code/src/resampling/systematic.py

Uniform samples:

$$u \sim \mathcal{U}(0, 1/N] \quad (109)$$

$$u_k = u + k/N, \quad k = 0, 1, \dots, N-1 \quad (110)$$

Selection via binary search on cumulative weights.

10.2 Soft Resampling

File: code/src/resampling/soft.py

Mixture distribution:

$$q(i) = \alpha w_i + \frac{1 - \alpha}{N} \quad (111)$$

New weights:

$$w'_i = \frac{w_i}{q(i)} \quad (112)$$

10.3 Entropy-Regularized Optimal Transport

File: code/src/resampling/ot_entropy.py

Cost matrix:

$$C_{ij} = \frac{\|x_i - y_j\|^2}{2} \quad (113)$$

Sinkhorn algorithm:

$$\text{softmax}_i(f) = -\varepsilon \log \left(\sum_j \exp \left(\frac{f_j - C_{ij}}{\varepsilon} \right) \right) \quad (114)$$

$$\alpha_i^{\text{new}} = \gamma \alpha_i + (1 - \gamma) \text{softmax}_i(\beta + \varepsilon \log w) \quad (115)$$

$$\beta_j^{\text{new}} = \gamma \beta_j + (1 - \gamma) \text{softmax}_j(\alpha) \quad (116)$$

Transport matrix:

$$T_{ij} = \exp \left(\frac{\alpha_i + \beta_j - C_{ij}}{\varepsilon} \right) w_i \quad (117)$$

11 Parameter Inference

11.1 HMC for Differentiable Filters

File: code/src/DF/hmc_runner.py

Log-posterior:

$$\log \pi(\theta) = \log p(y_{1:T}|\theta) + \log p(\theta) \quad (118)$$

Negative log-posterior (minimized by HMC):

$$U(\theta) = -\log \pi(\theta) \quad (119)$$

HMC Hamiltonian:

$$H(p, \theta) = \frac{1}{2}p^\top M^{-1}p + U(\theta) \quad (120)$$

Leapfrog integration:

$$p \leftarrow p - \frac{\varepsilon}{2} \nabla_\theta U(\theta) \quad (121)$$

$$\theta \leftarrow \theta + \varepsilon M^{-1}p \quad (122)$$

$$p \leftarrow p - \frac{\varepsilon}{2} \nabla_\theta U(\theta) \quad (123)$$

Metropolis acceptance:

$$\alpha = \min(1, \exp(-(H_{\text{end}} - H_{\text{start}}))) \quad (124)$$

Dual averaging step size adaptation:

$$w = \frac{1}{m + t_0} \quad (125)$$

$$H_{\text{bar}} \leftarrow (1 - w)H_{\text{bar}} + w(\delta - \text{accept_prob}) \quad (126)$$

$$\log \varepsilon \leftarrow \mu - \frac{\sqrt{m}}{\gamma} H_{\text{bar}} \quad (127)$$

11.2 Parameter Bijections

File: `code/src/DF/parameter_handler.py`

Change of variables:

$$\log p(\theta_{\text{unconstrained}}) = \log p(\theta_{\text{constrained}}) + \log |\det(J)| \quad (128)$$

Softplus (for $\theta > 0$):

$$\theta_{\text{constrained}} = \text{softplus}(\theta_{\text{unconstrained}}) = \log(1 + \exp(\theta_{\text{unconstrained}})) \quad (129)$$

Sigmoid (for $0 < \theta < 1$):

$$\theta_{\text{constrained}} = \frac{1}{1 + \exp(-\theta_{\text{unconstrained}})} \quad (130)$$

11.3 PMMH (Particle Marginal Metropolis-Hastings)

File: `code/src/DF/pmmh_runner.py`

Random walk proposal:

$$\theta^* = \theta + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \Sigma_{\text{proposal}}) \quad (131)$$

Acceptance probability:

$$\alpha = \min \left(1, \frac{\hat{p}(y|\theta^*)p(\theta^*)}{\hat{p}(y|\theta)p(\theta)} \right) \quad (132)$$

11.4 Particle Gibbs + HMC

File: code/src/DF/pgibbs_runner.py

Closed-form log-posterior (theta-step):

$$\log p(\theta|x, y) = \log p(x_0|\theta) + \sum_t \log p(x_t|x_{t-1}, \theta) + \sum_t \log p(y_t|x_t, \theta) + \log p(\theta) \quad (133)$$

12 Numerical Linear Algebra

12.1 Safe Cholesky

File: code/src/utils/linalg.py

Adaptive jitter:

$$L = \text{cholesky}(A + \text{jitter} \cdot \frac{\text{trace}(A)}{n} I) \quad (134)$$

12.2 Safe Log-Absolute Determinant

File: code/src/utils/linalg.py

Cholesky-based:

$$\log |\det(A)| = 2 \sum_i \log(L_{i,i}), \quad L = \text{cholesky}(A) \quad (135)$$

SVD-based (robust):

$$\log |\det(A)| = \sum_i \log(\max(s_i, 10^{-30})) \quad (136)$$

12.3 Matrix Square Root

File: code/src/utils/linalg.py

Via eigenvalue decomposition:

$$\sqrt{A} = V \text{diag}(\sqrt{\lambda_i}) V^\top \quad (137)$$

12.4 Symmetrize

File: code/src/utils/linalg.py

$$\text{sym}(A) = \frac{A + A^\top}{2} \quad (138)$$

13 ODE/SDE Solvers

13.1 Euler Method

File: code/src/utils/ode_solvers.py

$$x_{n+1} = x_n + f(x_n)\Delta t \quad (139)$$

13.2 Runge-Kutta 4

File: code/src/utils/ode_solvers.py

$$k_1 = f(x_n) \quad (140)$$

$$k_2 = f(x_n + \frac{\Delta t}{2}k_1) \quad (141)$$

$$k_3 = f(x_n + \frac{\Delta t}{2}k_2) \quad (142)$$

$$k_4 = f(x_n + \Delta t \cdot k_3) \quad (143)$$

$$x_{n+1} = x_n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (144)$$

13.3 Euler-Maruyama (SDE)

File: code/src/utils/ode_solvers.py

$$x_{n+1} = x_n + f(x_n)\Delta t + g(x_n)\sqrt{\Delta t} dW \quad (145)$$

14 LaTeX Report – Additional Derivations

14.1 Importance Sampling Discretization

File: report/main.tex

$$p(x_k|y_{1:k-1}) \approx \sum_i w_{k-1}^i p(x_k|x_{k-1}^i) \quad (146)$$

14.2 KL Divergence for Kernel Flow

File: report/main.tex

$$\text{KL}(q\|\pi) = \int q(x) \log \frac{q(x)}{\pi(x)} dx \quad (147)$$

14.3 Entropy-Regularized Wasserstein Distance

File: *report/main.tex*

$$W_{2,\varepsilon}^2 = \min_T \sum_{ij} T_{ij} \left(c_{ij} + \varepsilon \log \frac{T_{ij}}{a_i b_j} \right) \quad (148)$$

14.4 Joseph Form Limiting Behavior

File: *report/main.tex*

$$\lim_{R \rightarrow 0} P_{\text{updated}} = \frac{R}{h^2} \quad (149)$$

14.5 Fokker-Planck Equation

File: *report/main.tex*

$$\frac{\partial p}{\partial t} = -\nabla \cdot (fp) + \frac{1}{2} \nabla \cdot (Q \nabla p) \quad (150)$$

15 Summary of Formulas Requiring Hand Derivation

The following formulas are non-trivial and would typically require careful hand derivation to verify correctness before implementation:

1. **EDH flow parameters $A(\lambda)$ and $b(\lambda)$** – Derived from the log-homotopy equation by assuming a Gaussian ansatz and matching terms. Requires matrix calculus and completing the square in the exponent.
2. **LEDH local flow with nonlinear correction** – The drift $b(\lambda)$ includes a correction term $e = h(x) - Hx$ that accounts for linearization error. Must verify the expansion is consistent with the homotopy.
3. **Stochastic flow score correction** – Adding $\frac{1}{2} Q \nabla_x \log p$ to the deterministic drift requires deriving the score function of the intermediate Gaussian $p(x, \lambda)$ and verifying it satisfies the Fokker-Planck equation.
4. **Importance weight formula for invertible flows** – The full expression $\log w = \log p(z|\eta_1) + \log p(\eta_1|x_{k-1}) + \log \theta - \log p(\eta_0|x_{k-1})$ comes from the change-of-variables formula for the flow map. Requires tracking the Jacobian determinant through the Euler steps.
5. **Observation Jacobians for each model** – Each nonlinear model (range-bearing, acoustic, cubic sensor, Kitagawa, stochastic volatility) requires manual differentiation of $h(x)$ to obtain $H = \partial h / \partial x$.

6. **State Jacobians** – The Kitagawa model’s $\frac{df}{dx} = \frac{1}{2} + \frac{25(1-x^2)}{(1+x^2)^2}$ requires the quotient rule applied to $\frac{25x}{1+x^2}$.
7. **BVP for optimal homotopy schedule** – Deriving $\beta'' = \mu \frac{\partial \kappa}{\partial \beta}$ from the stiffness minimization objective, and computing $\frac{\partial \kappa_\nu}{\partial \beta}$ involving traces of matrix products and inverses.
8. **Stiffness ratio from flow Jacobian eigenvalues** – Computing $F(\beta) = -\frac{1}{2}M(\beta)^{-1}J_{\text{meas}} - \frac{Q}{2}M(\beta)$ and analyzing its eigenvalue spread.
9. **Sinkhorn optimal transport** – The dual formulation of entropy-regularized OT, derivation of the softmin updates, and the transport matrix formula from dual variables.
10. **HMC leapfrog symplecticity** – Verifying the leapfrog integrator preserves the symplectic structure and that the Metropolis correction gives the correct stationary distribution.
11. **Bijector log-determinant Jacobians** – For the change-of-variables $\log p(\theta_{\text{unc}}) = \log p(\theta_{\text{con}}) + \log |\det J|$, each bijector (softplus, sigmoid) requires computing $\frac{d\theta_{\text{con}}}{d\theta_{\text{unc}}}$.
12. **Kernel flow SVGD gradient** – The Stein variational gradient $I_f = \frac{1}{N} \sum_j [K \nabla \log p + \nabla K]$ comes from minimizing KL divergence in the RKHS. Requires functional analysis / Stein’s identity.
13. **Joseph form covariance update** – Showing $(I - KH)P(I - KH)^\top + K R K^\top$ is equivalent to $(I - KH)P$ but numerically superior, and its limiting behavior as $R \rightarrow 0$.
14. **UKF sigma point weights** – Deriving the mean and covariance weights from the unscented transform moment-matching conditions.
15. **PMMH pseudo-marginal correctness** – Proving that using the particle filter’s marginal likelihood estimate $\hat{p}(y|\theta)$ in the MH ratio still targets the correct posterior $p(\theta|y)$.
16. **Particle Gibbs closed-form conditional** – Factoring the joint $p(\theta, x|y)$ to obtain $p(\theta|x, y)$ as a product of transition and observation densities conditioned on the sampled trajectory.